CPSC-406 Report

Andrew Floyd
Chapman University

March 5, 2026

Abstract

Contents

1 Introduction **1**

2 Week by Week **1**
2.1 Week 1 . . . 1
2.1.1 Exercise 1: Word Processing with DFAs . . . 1
2.1.2 Exercise 2: Designing DFAs . . . 3

3 Synthesis **5**
3.1 Week 2 . . . 5
3.1.1 Exercise 1: Product Automata . . . 5
3.1.2 Exercise 2: More Automata . . . 7
3.2 Week 3 . . . 8
3.2.1 Problem 1 . . . 8
3.2.2 Problem 2 . . . 11
3.3 Week 4 . . . 12
3.3.1 Exercise 1: Kleene's Algorithm . . . 12

4 Evidence of Participation **14**

5 Conclusion **14**

1 Introduction

2 Week by Week

2.1 Week 1

2.1.1 Exercise 1: Word Processing with DFAs

Notes and Homework:

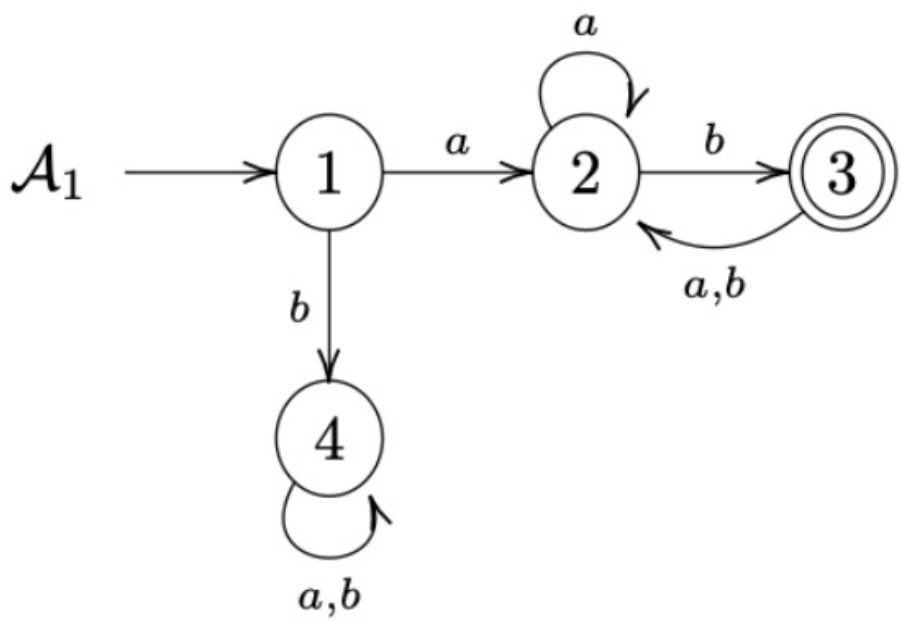

Figure 1: Automata A_1 and A_2

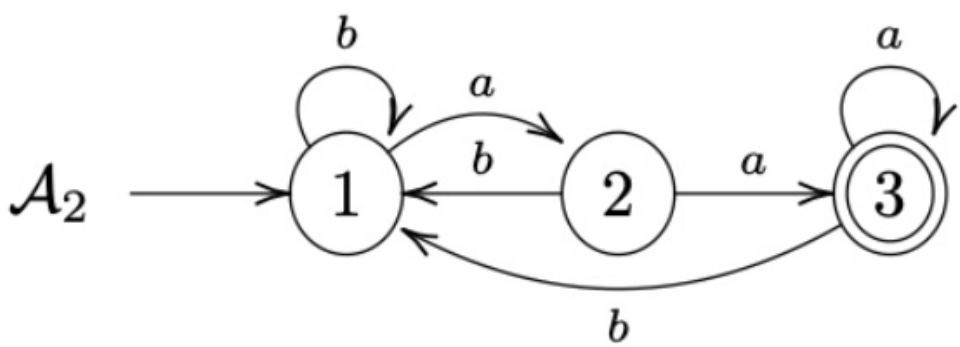

Figure 2: Additional automata reference

Question 1: Which of the following words are accepted/refused by A_1 and A_2, respectively? Complete the table.

w	accepted by A_1?	accepted by A_2?
aaa	no	yes
aab	yes	no
aba	no	no
abb	no	no
baa	no	yes
bab	no	no
bba	no	no
bbb	no	no

Table 1: Acceptance table for A_1 and A_2

Question 2: More generally, can you completely describe the languages $L(A_k)$ accepted by A_k, for $k = 1, 2$?

The language $L(A)$ accepted by this DFA is:

$$L(A) = \{w \in \{a, b\}^* \mid \text{the number of } a\text{'s in } w \text{ is even}\}$$

2.1.2 Exercise 2: Designing DFAs

Design DFAs whose accepted languages are given as follows:

1. All the words that end with ab
2. All the words that contain ab
3. All the words that contain an odd number of a's and an odd number of b's
4. All the words that contain an even number of a's and an odd number of b's
5. All the words such that any three consecutive characters contain at least one a
6. All the words that contain aba

DFA Diagrams:

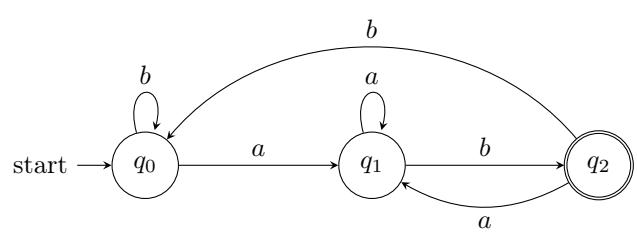

Figure 3: DFA for words ending with ab

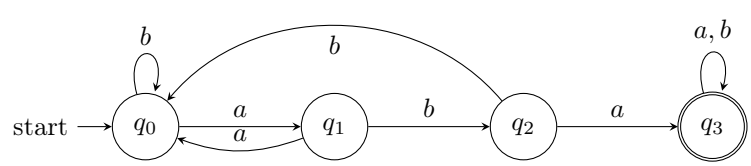

Figure 4: DFA for words containing aba

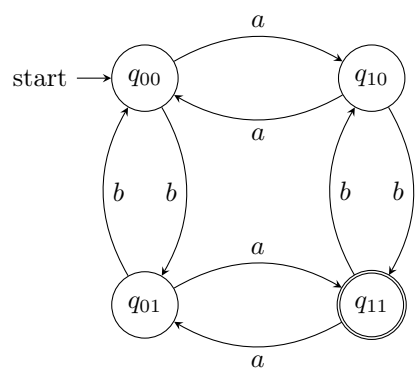

Figure 5: DFA for words with odd number of a's and odd number of b's

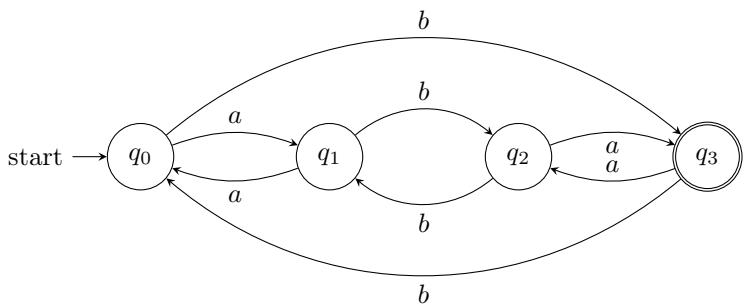

Figure 6: DFA for words with even number of a's and odd number of b's

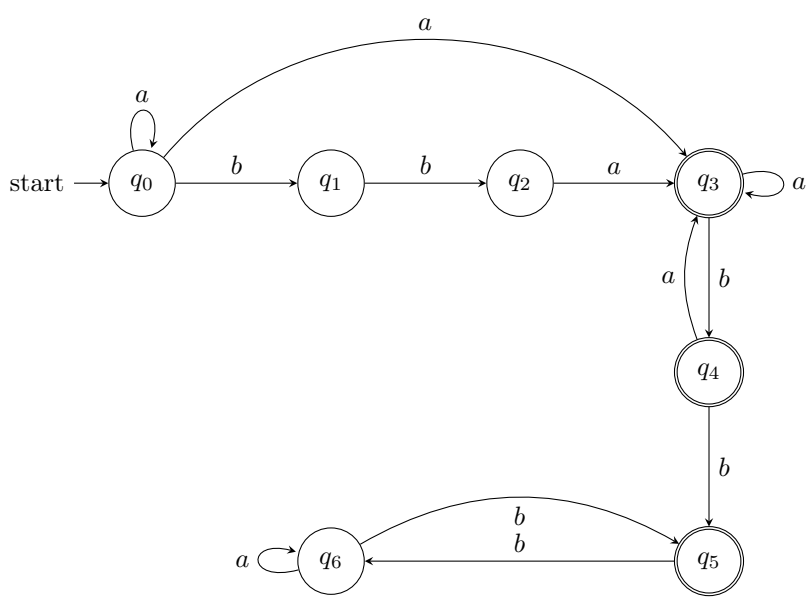

Figure 7: DFA for words where any three consecutive characters contain at least one a

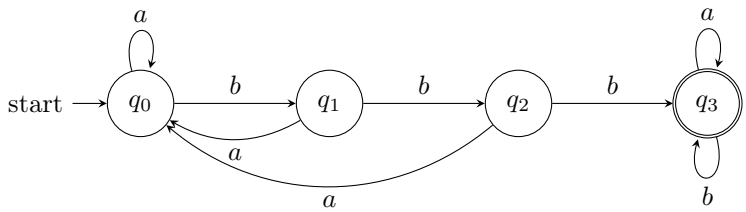

Figure 8: DFA for words containing bbb

What do you notice when comparing the various automata?

When comparing the DFAs designed above, several patterns emerge:

- **State count reflects language complexity:** Simple properties like "ends with ab" require only 3 states, while tracking multiple independent conditions (like counting both a's and b's modulo 2) requires 2^n states where n is the number of independent conditions being tracked.
- **Parity-based languages form grid structures:** DFAs for languages involving "odd/even" counts tend to have states arranged in a grid pattern, where each dimension corresponds to one parity condition. The transitions toggle between states along each axis.
- **Substring containment uses linear chains:** DFAs that check for substring containment (like "contains ab" or "contains bbb") have a linear chain of states tracking progress toward finding the substring, with "reset" transitions back to earlier states when the pattern is broken.
- **Accepting states indicate language structure:** For "contains" languages, once the target is found, we stay in an accepting sink state. For parity languages, specific combinations of parities determine acceptance.
- **Self-loops handle "don't care" inputs:** When an input symbol doesn't affect progress toward acceptance (like a's in "contains bbb" after reaching the accepting state), self-loops keep the automaton in place.

3 Synthesis

3.1 Week 2

3.1.1 Exercise 1: Product Automata

Homework prompt: Work with the two DFAs shown below, describe their languages, and build the product automata needed for intersection and union.

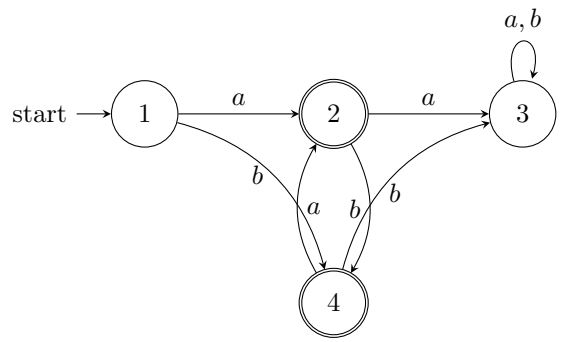

Figure 9: Automaton $\mathcal{A}^{(1)}$

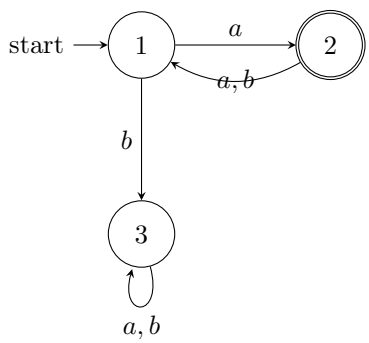

Figure 10: Automaton $\mathcal{A}^{(2)}$

Question 1: Describe $L(\mathcal{A}^{(1)})$ and $L(\mathcal{A}^{(2)})$.

- $L(\mathcal{A}^{(1)})$ is "all strings that stop the moment you land in state 2 or jump from 2 to 4." You must reach 2 by reading an a. You may then loop through "a back to 2, b down to 4" as often as you like, but the word has to end right after that last a into 2 or b into 4.
- $L(\mathcal{A}^{(2)})$ is "all strings that start with a, never fall into state 3, and stay parked in 2." Once you enter 2 on the first a, every later letter must be whatever keeps you there; if you ever take the b to 3 the word is rejected.

Question 2: Construct the intersection automaton $\mathcal{A}$.

The standard product construction uses ordered pairs (p, q) where p is a state of $\mathcal{A}^{(1)}$ and q a state of $\mathcal{A}^{(2)}$. We only draw reachable states.

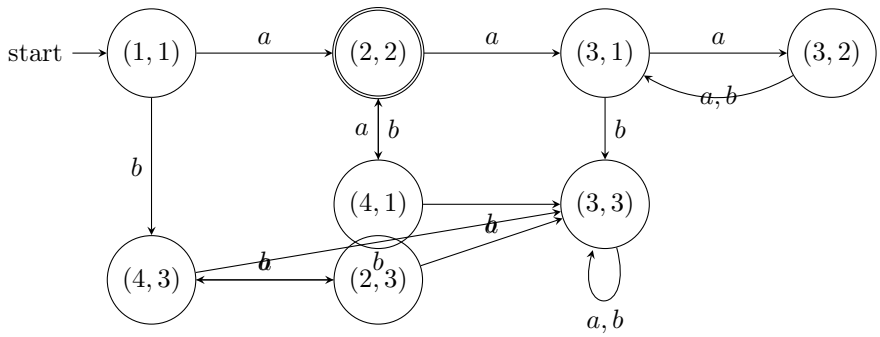

Figure 11: Intersection automaton $\mathcal{A}$

Question 3: Show $L(\mathcal{A}) = L(\mathcal{A}^{(1)}) \cap L(\mathcal{A}^{(2)})$.

The product DFA only has one double-circle state, $(2, 2)$. Getting there means the $\mathcal{A}^{(1)}$ copy is sitting happily in 2 (after possibly bouncing through 4) while the $\mathcal{A}^{(2)}$ copy is also in 2. So a word survives in the product exactly when it makes both original machines happy at the same time, i.e., when it lies in the intersection.

Question 4: Modify $\mathcal{A}$ to obtain $\mathcal{A}'$ with $L(\mathcal{A}') = L(\mathcal{A}^{(1)}) \cup L(\mathcal{A}^{(2)})$.

Keep the same transition structure but change the accepting set to

$$F' = \{(p, q) \mid p \in F_{\mathcal{A}^{(1)}} \text{ or } q \in F_{\mathcal{A}^{(2)}}\}.$$

Now any word accepted by either original automaton drives the product to a state whose pair has at least one accepting component, so the language is the union.

3.1.2 Exercise 2: More Automata

Homework prompt: Repeat the same analysis for the DFAs $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ below.

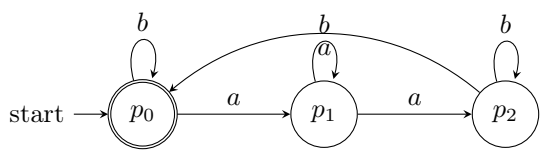

Figure 12: Automaton $\mathcal{B}^{(1)}$

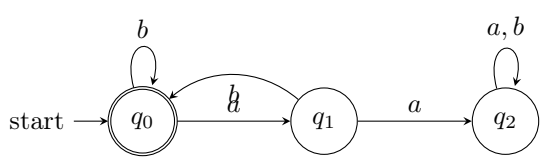

Figure 13: Automaton $\mathcal{B}^{(2)}$

Question 1: Describe $L(\mathcal{B}^{(1)})$ and $L(\mathcal{B}^{(2)})$.

- $L(\mathcal{B}^{(1)})$ = "count a's in groups of three and ignore b's." The machine walks through $p_0 \to p_1 \to p_2 \to p_0$ every time an a shows up, so you only end in the accepting home state p_0 when the total number of a's is a multiple of three.
- $L(\mathcal{B}^{(2)})$ = "never write two a's back-to-back." You can stay in q_0 on b's, hop to q_1 on a single a, but a second immediate a throws you into the sink q_2. As long as every a is separated by at least one b, you return to q_0 and the word is accepted.

Question 2: Construct the intersection automaton $\mathcal{B}$.

The full product has nine reachable states. Starting from (p_0, q_0) we obtain the transition graph shown below. Because only p_0 and q_0 are accepting in their respective machines, the sole accepting pair is (p_0, q_0).

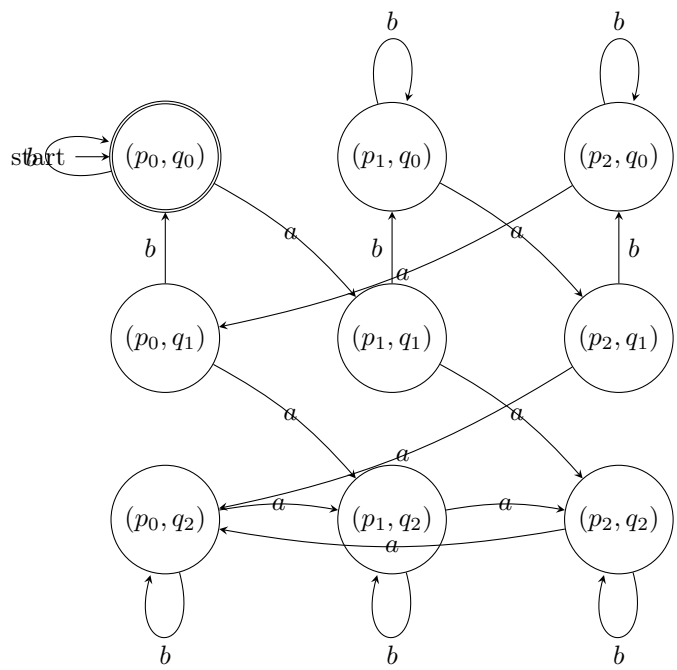

Figure 14: Intersection automaton $\mathcal{B}$

Question 3: Why does $L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)})$?

Each arrow in the product just mirrors the arrows in the two base DFAs. The only double-circle state is (p_0, q_0), so the combined machine says "yes" exactly when both halves also say "yes" at the end of the word. That is the definition of the intersection.

Question 4: Obtaining the union language.

As in Exercise 1, we can apply De Morgan's law. Complement each DFA by flipping accepting/non-accepting states, build the product, and complement again. Equivalently, start from the product above and simply mark as accepting every pair where at least one component is accepting. Either construction yields an automaton $\mathcal{B}'$ whose language is $L(\mathcal{B}^{(1)}) \cup L(\mathcal{B}^{(2)})$.

3.2 Week 3

3.2.1 Problem 1

Homework prompt: For the alphabet $\Sigma = \{a, b, c\}$, consider the following NFA $\mathcal{A}$:

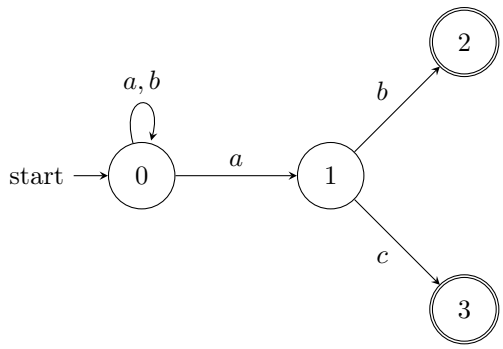

Figure 15: NFA $\mathcal{A}$ over $\Sigma = \{a, b, c\}$

Question 1: Describe the language accepted by $\mathcal{A}$ using a regular expression.

State 0 loops on a and b, so we can read any mix of a's and b's at the start. At some point we take the a-arrow into state 1, and then read one more symbol: b lands in state 2, or c lands in state 3. Both are accepting with no way out, so the word ends right there. In short, every accepted word is "some a's and b's, then a, then b or c":

$$L(\mathcal{A}) \;=\; (a+b)^*\, a\, (b+c).$$

Question 2: Specify the NFA $\mathcal{A}$ in the form $(Q, \Sigma, \delta, q_0, F)$.

$$\mathcal{A} = (Q,\ \Sigma,\ \delta,\ q_0,\ F)$$

where

- $Q = \{0, 1, 2, 3\}$
- $\Sigma = \{a, b, c\}$
- $q_0 = 0$
- $F = \{2, 3\}$
- The transition function δ (which maps each state–symbol pair to a set of possible next states) is:

δ	a	b	c
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\{3\}$
2	$\emptyset$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

Table 2: Transition function of $\mathcal{A}$

Question 3: Find all paths in $\mathcal{A}$ for the word $v = abab$.

We follow every possible way the NFA can read $abab$ from start to finish. Paths that get stuck before reading the whole word are left out:

$$0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 0 \xrightarrow{b} 0 \quad \text{(not accepting)}$$

$$0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \quad \textbf{(accepting ✓)}$$

Figure 16: All complete paths in $\mathcal{A}$ for $v = abab$

Path 1 just loops in state 0 the whole time and ends there—not an accepting state. Path 2 jumps to state 1 on the third letter (a) and then lands in the accepting state 2 on the fourth letter (b). There is also a branch $0 \xrightarrow{a} 1 \xrightarrow{b} 2$ that dies after only two symbols (state 2 has no arrows out), so we leave it out. Because at least one path reaches an accepting state, $v = abab$ is **accepted**.

Question 4: Construct the determinization $\mathcal{A}^D$ using the power set construction.

The idea is to track which NFA states could be active at the same time. Each DFA state is a *set* of NFA states. Starting from $\{0\}$, we build the table row by row, only adding sets we haven't seen yet:

State	a	b	c	Accepting?
$\{0\}$	$\{0,1\}$	$\{0\}$	$\emptyset$	No
$\{0,1\}$	$\{0,1\}$	$\{0,2\}$	$\{3\}$	No
$\{0,2\}$	$\{0,1\}$	$\{0\}$	$\emptyset$	Yes
$\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$	Yes
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	No

Table 3: Transition table for $\mathcal{A}^D$

We start in $\{0\}$. A set is accepting whenever it contains an original accepting state (2 or 3), so the accepting states are $\{0,2\}$ and $\{3\}$.

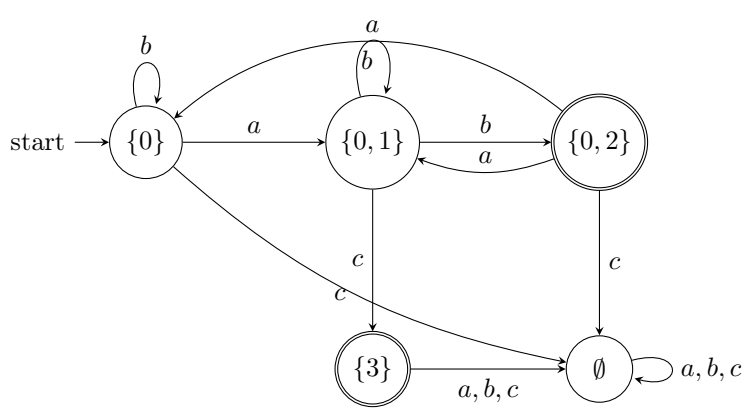

Figure 17: Determinized DFA $\mathcal{A}^D$

3.2.2 Problem 2

Homework prompt: For the alphabet $\Sigma = \{0, 1\}$, consider the following NFA $\mathcal{A}$:

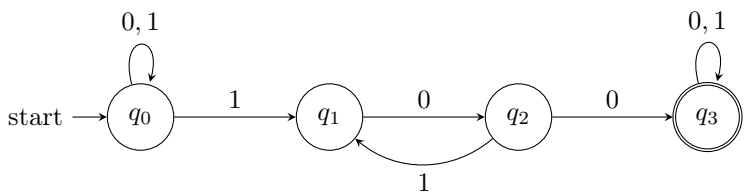

Figure 18: NFA $\mathcal{A}$ over $\Sigma = \{0, 1\}$

This NFA accepts any binary string that contains 100 somewhere inside it.

Question 1: Construct the determinization $\mathcal{A}^D$ using the power set construction.

Again we track which NFA states can be active simultaneously. Starting from $\{q_0\}$:

State	0	1	Accepting?
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$	No
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1\}$	No
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1\}$	No
$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$	Yes
$\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	Yes
$\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$	Yes

Table 4: Transition table for $\mathcal{A}^D$

We start in $\{q_0\}$. Any set containing q_3 is accepting: $\{q_0, q_3\}$, $\{q_0, q_1, q_3\}$, and $\{q_0, q_2, q_3\}$.

To keep the diagram readable, we use short names: $A = \{q_0\}$, $B = \{q_0, q_1\}$, $C = \{q_0, q_2\}$, $D = \{q_0, q_3\}$, $E = \{q_0, q_1, q_3\}$, $F = \{q_0, q_2, q_3\}$.

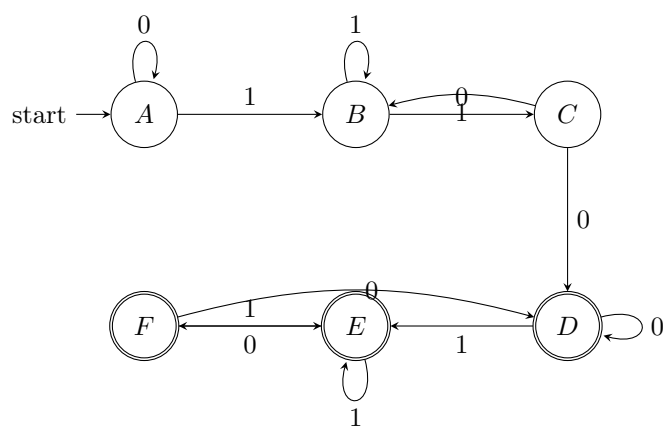

Figure 19: Determinized DFA $\mathcal{A}^D$ (6 states)

Question 2: Is there a smaller DFA accepting the same language as $\mathcal{A}^D$?

Yes. We can merge states that behave identically. The basic idea: start by grouping states into "accepting" vs. "non-accepting," then keep splitting any group whose members disagree on where they go.

Step 1. Two initial groups: $\{A, B, C\}$ (non-accepting) and $\{D, E, F\}$ (accepting).

Step 2. Look at $\{A, B, C\}$: on input 0, both A and B stay in the non-accepting group, but C jumps to D (accepting). So C doesn't belong with the others. New groups: $\{A, B\}$, $\{C\}$, $\{D, E, F\}$.

Step 3. Look at $\{A, B\}$: on input 0, A goes to A (same group) but B goes to C (different group). Split again: $\{A\}$, $\{B\}$. The accepting group $\{D, E, F\}$ needs no splitting—all three states go to the same groups on every input.

Step 4. Nothing left to split. Final groups: $\{A\}$, $\{B\}$, $\{C\}$, $\{D, E, F\}$.

Each group becomes one state in the minimal DFA, giving us **4 states**:

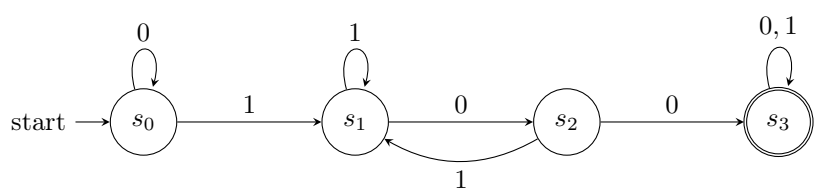

Figure 20: Minimal DFA for $L(\mathcal{A})$ (4 states)

Each state has a simple meaning:

- s_0: haven't started matching 100 yet.
- s_1: just read a 1 (first character of 100).
- s_2: just read 10 (two characters matched).
- s_3: already saw 100—accept everything from here on.

No further merging is possible because each non-accepting state reacts differently to at least one input, so 4 states is the smallest this DFA can be.

3.3 Week 4

3.3.1 Exercise 1: Kleene's Algorithm

Homework prompt: Consider the following DFA with states p and q:

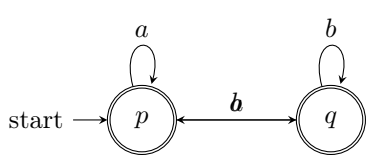

Figure 21: DFA for Kleene's algorithm exercise

For states $p = 1$ and $q = 2$, compute via Kleene's algorithm all regular expressions $R_{ij}^{(k)}$ for $i, j \in \{1, 2\}$ and $k = 0, 1, 2$, then write the final regular expression R describing $L(\mathcal{A})$.

Solution using Kleene's Algorithm:

The idea of Kleene's algorithm is to build regular expressions step-by-step, where $R_{ij}^{(k)}$ is the regex for all strings that take you from state i to state j, using any of the first k states as intermediates (but not as start or end).

Step 0: Direct transitions (no intermediates allowed)

These are just the direct arrows in the diagram:

	$\to 1$	$\to 2$
From 1	a	b
From 2	a	b

Table 5: Base case: $R_{ij}^{(0)}$

Formally:

- $R_{11}^{(0)} = a$
- $R_{12}^{(0)} = b$
- $R_{21}^{(0)} = a$
- $R_{22}^{(0)} = b$

Step 1: Using state 1 as an intermediate

Now we can loop through state 1. The formula is: $R_{ij}^{(1)} = R_{ij}^{(0)} \mid R_{i1}^{(0)}(R_{11}^{(0)})^* R_{1j}^{(0)}$.

This means: either take the direct path, or go to state 1, loop there as much as you want, then leave toward state j.

- $R_{11}^{(1)} = a \mid a(a)^*a = a(1 \mid a^*) = a^+$
- $R_{12}^{(1)} = b \mid a(a)^*b = b \mid a^+b = a^*b$ (either b directly, or loop at 1 first)
- $R_{21}^{(1)} = a \mid a(a)^*a = a^+$
- $R_{22}^{(1)} = b \mid a(a)^*b = a^*b$

Step 2: Using states 1 and 2 as intermediates

Now we can use both states. The formula is: $R_{ij}^{(2)} = R_{ij}^{(1)} \mid R_{i2}^{(1)}(R_{22}^{(1)})^* R_{2j}^{(1)}$.

This means: either use the paths from Step 1, or detour through state 2 (looping there as much as you want).

- $R_{11}^{(2)} = a^+ \mid (a^*b)(a^*b)^*(a^+) = a^+ \mid a^*b(a^*b)^*a^+$
- $R_{12}^{(2)} = a^*b \mid (a^*b)(a^*b)^*(a^+) = a^*b(1 \mid (a^*b)^*a^+)$
- $R_{21}^{(2)} = a^+ \mid (a^*b)(a^*b)^*(a^+) = a^+ \mid a^*b(a^*b)^*a^+$
- $R_{22}^{(2)} = a^*b \mid (a^*b)(a^*b)^*(a^+) = a^*b(1 \mid (a^*b)^*a^+)$

Final Language:

Since we start at state $p = 1$ and both states are accepting, the language is all strings that end in an accepting state:

$$L(\mathcal{A}) = R_{11}^{(2)} \mid R_{12}^{(2)} \mid \varepsilon$$

where ε is included because state 1 is accepting and it's the starting state.

Substituting:

$$L(\mathcal{A}) = (a^+ \mid a^*b(a^*b)^*a^+) \mid a^*b \mid \varepsilon$$

Simplifying by grouping:

$$L(\mathcal{A}) = \varepsilon \mid a^+ \mid a^*b \mid a^*b(a^*b)^*a^+$$

Interpretation:

This regular expression describes four types of accepted strings:

1. ε: the empty string (start at state 1, which is accepting)
2. a^+: one or more a's (loop at state 1)
3. a^*b: any number of a's, then b (reach state 2, which is accepting)
4. $a^*b(a^*b)^*a^+$: any number of (a^*b) pairs, then at least one a (repeatedly bounce between states via the $1 \xrightarrow{b} 2$ and $2 \xrightarrow{a} 1$ path)

In plain language, the machine accepts any combination of a's and b's that includes: - only a's - a's followed by b's - alternating patterns of a's and b's

Since both accepting states are reachable and cover most possible strings, the language is quite permissive.

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.